



{230}

A. M. Turing

[NOV. 12 1936.]

[Received 28 May,
1936.—Read 12
November, 1936.]

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO THE ENTSCHEIDUNGSPROBLEM

By A. M. TURING

There are many complex characters in this paper; if you find them difficult to distinguish, you are advised to increase the viewing size. (In IExplorer, go to View menu, Text Size; in Netscape, go to View menu, Increase Font.)

INDEX



1. [Computing machines.](#)
2. [Definitions.](#)
 - [Automatic machines.](#)
 - [Computing machines.](#)
 - [Circle and circle-free numbers.](#)
 - [Computable sequences and numbers.](#)
3. [Examples of computing machines.](#)
4. [Abbreviated tables](#)
 - [Further examples.](#)
5. [Enumeration of computable sequences.](#)
6. [The universal computing machine.](#)
7. [Detailed description of the universal machine.](#)
8. [Application of the diagonal process.](#)

[9. The extent of the computable numbers.](#)

[10. Examples of large classes of numbers which are computable.](#)

[11. Application to the Entscheidungsproblem.](#)

[APPENDIX](#)

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO THE ENTSCHEIDUNGSPROBLEM.

[A CORRECTION](#) By A. M. Turing

[Publisher's copyright notice—the London Mathematical Society](#)

[Endnotes](#)

Ads by Google

[CSS Standards Web Design](#)

Art & Logic Inc. - Software and Web application development since 1991.
www.artlogic.com/web

[CSS Style Sheet Users](#)

Get 800 MB storage, 40 GB transfer, PHP, Perl, MySQL \$7.95/mo. (aff.)
www.lunarpages.com



[advertising disclaimer](#)

The “computable” numbers may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means. Although the subject of this paper is ostensibly the computable *numbers*, it is almost equally easy to define and investigate computable functions of an integral variable or a real or computable variable, computable predicates, and so forth. The fundamental problems involved are, however, the same in each case, and I have chosen the computable numbers for explicit treatment as involving the least cumbersome technique. I hope shortly to give an account of the relations of the computable numbers, functions, and so forth to one another. This will include a development of the theory of functions of a real variable expressed in terms of computable numbers. According to my definition, a number is computable if its decimal can be written down by a machine.

In §§ [9](#), [10](#) I give some arguments with the intention of showing that the computable numbers include all numbers which could naturally be regarded as computable. In particular, I show that certain large classes of numbers are computable. They include, for instance, the real parts of all algebraic numbers, the real parts of the zeros of the Bessel

functions, the numbers π , e , etc. The computable numbers do not, however, include all definable numbers, and an example is given of a definable number which is not computable.

Although the class of computable numbers is so great, and in many ways similar to the class of real numbers, it is nevertheless enumerable. In §8 I examine certain arguments which would seem to prove the contrary. By the correct application of one of these arguments, conclusions are reached which are superficially similar to those of Gödel [1]. These results {231} have valuable applications. In particular, it is shown (§11) that the Hilbertian Entscheidungsproblem can have no solution.

In a recent paper Alonzo Church[2] has introduced an idea of “effective calculability”, which is equivalent to my “computability”, but is very differently defined. Church also reaches similar conclusions about the Entscheidungsproblem.[3] The proof of equivalence between “computability” and “effective calculability” is outlined in an appendix to the present paper.



1. Computing machines.

We have said that the computable numbers are those whose decimals are calculable by finite means. This requires rather more explicit definition. No real attempt will be made to justify the definitions given until we reach §9. For the present I shall only say that the justification lies in the fact that the human memory is necessarily limited.

We may compare a man in the process of computing a real number to a machine which is only capable of a finite number of conditions $q_1, q_2, \dots, q_R$ which will be called “ m -configurations”. The machine is supplied with a “tape”, (the analogue of paper) running through it, and divided into sections (called “squares”) each capable of bearing a “symbol”. At any moment there is just one square, say the r -th, bearing the symbol $\mathfrak{S}(r)$ which is “in the machine”. We may call this square the “scanned square”. The symbol on the scanned square may be called the “scanned symbol”. The “scanned symbol” is the only one of which the machine is, so to speak, “directly aware”. However, by altering its m -configuration the machine can effectively remember some of the symbols which it has “seen” (scanned) previously. The possible behaviour of the machine at any moment is determined by the m -configuration q_n and the scanned symbol $\mathfrak{S}(r)$. This pair $q_n, \mathfrak{S}(r)$ will be called the “configuration”: thus the configuration determines the possible behaviour of the machine. In some of the configurations in which the scanned square is blank (i.e. bears no symbol) the machine writes down a new symbol on the scanned square: in other configurations it erases the scanned symbol. The machine may also change the square which is being scanned, but only by shifting it one place to right or left. In addition to any of these operations the m -configuration may be changed. Some of the symbols written down {232} will form the sequence of figures which is the decimal of the real number which is being computed. The others are just rough notes to “assist the memory”. It will only be these rough notes which will be liable to erasure.

It is my contention that these operations include all those which are used in the computation of a number. The defence of this contention will be easier when the theory of the machines is familiar to the reader. In the next section I therefore proceed with the development of the theory and assume that it is understood what is meant by “machine”, “tape”, “scanned”, etc.



2. Definitions.

Automatic machines.

If at each stage the motion of a machine (in the sense of §1) is *completely* determined by the configuration, we shall call the machine an “automatic machine” (or *a-machine*). For some purposes we might use machines (choice machines or *c-machines*) whose motion is only partially determined by the configuration (hence the use of the word “possible” in §1). When such a machine reaches one of these ambiguous configurations, it cannot go on until some arbitrary choice has been made by an external operator. This would be the case if we were using machines to deal with axiomatic systems. In this paper I deal only with automatic machines, and will therefore often omit the prefix *a-*.

Computing machines.

If an *a-machine* prints two kinds of symbols, of which the first kind (called figures) consists entirely of 0 and 1 (the others being called symbols of the second kind), then the machine will be called a computing machine. If the machine is supplied with a blank tape and set in motion, starting from the correct initial *m*-configuration, the subsequence of the symbols printed by it which are of the first kind will be called the *sequence computed by the machine*. The real number whose expression as a binary decimal is obtained by prefacing this sequence by a decimal point is called the *number computed by the machine*.

At any stage of the motion of the machine, the number of the scanned square, the complete sequence of all symbols on the tape, and the *m*-configuration will be said to describe the *complete configuration* at that stage. The changes of the machine and tape between successive complete configurations will be called the *moves* of the machine.

{233}

Circular and circle-free machines.

If a computing machine never writes down more than a finite number of symbols of the first kind it will be called *circular*. Otherwise it is said to be *circle-free*.

A machine will be circular if it reaches a configuration from which there is no possible move, or if it goes on moving, and possibly printing symbols of the second kind, but cannot print any more symbols of the first kind. The significance of the term “circular” will be explained in §8.

Computable sequences and numbers.

A sequence is said to be computable if it can be computed by a circle-free machine. A number is computable if it differs by an integer from the number computed by a circle-free machine.

We shall avoid confusion by speaking more often of computable sequences than of computable numbers.



3. Examples of computing machines.

I. A machine can be constructed to compute the sequence 010101.... The machine is to have the four m -configurations “**b**”, “**c**”, “**f**”, “**e**” and is capable of printing “0”, and “1”. The behaviour of the machine is described in the following table in which “ R ” means “the machine moves so that it scans the square immediately on the right of the one it was scanning previously”. Similarly for “ L ”. “ E ” means the scanned symbol is “erased” and “ P ” stands for “prints”. This table (and all succeeding tables of the same kind) is to be understood to mean that for a configuration described in the first two columns the operations in the third column are carried out successively, and the machine then goes over into the m -configuration described in the last column. When the second column is left blank, it is understood that the behaviour of the third and fourth columns applies for any symbol and for no symbol. The machine starts in the m -configuration **b** with a blank tape.

<i>Configuration</i>		<i>Behaviour</i>	
b	None	$P0, R$	c
c	None	R	e
e	None	$P1, R$	f
f	None	R	b

{234} If (contrary to the description in §1) we allow the letters L, R to appear more than once in the operations column we can simplify the table considerably.

<i>m-</i> <i>config.</i>	<i>symbol</i>	<i>operations</i>	<i>final m-</i> <i>config.</i>
b {	None	$P0$	b
	0	$R, R, P1$	b
	1	$R, R, P0$	b

II. As a slightly more difficult example we can construct a machine to compute the sequence 00101101110111101111.... The machine is to be capable of five m -configurations, viz. “**o**”, “**q**”, “**p**”, “**f**”, “**b**” and of printing “**o**”, “**x**”, “0”, “1”. The first three symbols on the tape will be “**o o 0**”; the other figures follow on alternate squares. On the intermediate squares we never print anything but “**x**”. These letters serve to “keep the place” for us and are erased when we have finished with them. We also arrange that in the sequence of figures on alternate squares there shall be no blanks.

<i>Configuration</i>		<i>Behaviour</i>	
<i>m-</i> <i>config.</i>	<i>symbol</i>	<i>operations</i>	<i>final m-</i> <i>config.</i>
b		$Po, R, Po, R, P0,$ $R, R, P0, L, L$	o
o {	1	R, Px, L, L, L	o
	0		q
q	Any (0 or 1)	R, R	q

	$\left\{ \begin{array}{l} \text{None} \\ \end{array} \right.$	$P1, L$	$\mathfrak{p}$
$\mathfrak{p}$	$\left\{ \begin{array}{l} x \\ \mathfrak{a} \\ \text{None} \end{array} \right.$	$\begin{array}{l} E, R \\ R \\ L, L \end{array}$	$\begin{array}{l} \mathfrak{q} \\ \mathfrak{f} \\ \mathfrak{p} \end{array}$
$\mathfrak{f}$	$\left\{ \begin{array}{l} \text{Any} \\ \text{None} \end{array} \right.$	$\begin{array}{l} R, R \\ P0, L, L \end{array}$	$\begin{array}{l} \mathfrak{f} \\ \mathfrak{o} \end{array}$

To illustrate the working of this machine a table is given below of the first few complete configurations. These complete configurations are described by writing down the sequence of symbols which are on the tape, {235} with the m -configuration written below the scanned symbol. The successive complete configurations are separated by colons.

$\mathfrak{b}$	$\mathfrak{o}$	$\mathfrak{q}$	$\mathfrak{q}$	$\mathfrak{q}$	$\mathfrak{p}$			
$\mathfrak{a} \mathfrak{a} 0$	0	$1 : \mathfrak{a} \mathfrak{a} 0$	0	$1 : \mathfrak{a} \mathfrak{a} 0$	0	$1 :$		
$\mathfrak{p}$	$\mathfrak{p}$	$\mathfrak{f}$	$\mathfrak{f}$	$\mathfrak{o}$				
$\mathfrak{a} \mathfrak{a} 0$	0	$1 : \mathfrak{a} \mathfrak{a} 0$	0	1	$: \mathfrak{a} \mathfrak{a} 0$	0	1	$0 :$
$\mathfrak{a} \mathfrak{a} 0$	0	$1x 0 : \dots$						
$\mathfrak{o}$								

This table could also be written in the form

$$\mathfrak{b} : \mathfrak{a} \mathfrak{a} \mathfrak{o} 0 \quad 0 : \mathfrak{a} \mathfrak{a} \mathfrak{q} 0 \quad 0 : \quad \dots, \quad (C)$$

in which a space has been made on the left of the scanned symbol and the m -configuration written in this space. This form is less easy to follow, but we shall make use of it later for theoretical purposes.

The convention of writing the figures only on alternate squares is very useful: I shall always make use of it. I shall call the one sequence of alternate squares F -squares and the other sequence E -squares. The symbols on E -squares will be liable to erasure. The symbols on F -squares form a continuous sequence. There are no blanks until the end is reached. There is no need to have more than one E -square between each pair of F -squares: an apparent need of more E -squares can be satisfied by having a sufficiently rich variety of symbols capable of being printed on E -squares. If a symbol β is on an F -square S and a symbol α is on the E -square next on the right of S , then S and β will be said to be marked with α . The process of printing this α will be called marking β (or S) with α .



4. Abbreviated tables

There are certain types of process used by nearly all machines, and these, in some machines, are used in many connections. These processes include copying down sequences of symbols, comparing sequences, erasing all symbols of a given form, etc. Where such processes are concerned we can abbreviate the tables for the m -configurations considerably by the use of “skeleton tables”. In skeleton tables there appear capital German letters [4] and small Greek letters. These are of the nature of “variables”. By replacing each capital German letter throughout by an m -configuration {236} and each small Greek letter by a symbol, we obtain the table for an m -configuration.

The skeleton tables are to be regarded as nothing but abbreviations: they are not essential. So long as the reader understands how to obtain the complete tables from the skeleton tables, there is no need to give any exact definitions in this connection.

Let us consider an example:

m -configuration	Symbol	Behaviour	Final m -config.	
$\mathfrak{f}(\mathbf{C}, \mathfrak{B}, \alpha)$	$\left\{ \begin{array}{l} \mathfrak{a} \\ \text{not } \mathfrak{a} \end{array} \right.$	$\left\{ \begin{array}{l} L \\ L \end{array} \right.$	$\mathfrak{f}_1(\mathbf{C}, \mathfrak{B}, \alpha)$ $\mathfrak{f}(\mathbf{C}, \mathfrak{B}, \alpha)$	From the m -configuration $\mathfrak{f}(\mathbf{C}, \mathfrak{B}, \alpha)$ the machine
$\mathfrak{f}_1(\mathbf{C}, \mathfrak{B}, \alpha)$	$\left\{ \begin{array}{l} \alpha \\ \text{not } \alpha \\ \text{None} \end{array} \right.$	$\left\{ \begin{array}{l} R \\ R \end{array} \right.$	$\mathbf{C}$ $\mathfrak{f}_1(\mathbf{C}, \mathfrak{B}, \alpha)$ $\mathfrak{f}_2(\mathbf{C}, \mathfrak{B}, \alpha)$	finds the symbol of form α which is farthest to the left (the “first α ”) and the m -configuration then becomes $\mathbf{C}$. If there is
$\mathfrak{f}_2(\mathbf{C}, \mathfrak{B}, \alpha)$	$\left\{ \begin{array}{l} \alpha \\ \text{not } \alpha \\ \text{None} \end{array} \right.$	$\left\{ \begin{array}{l} R \\ R \end{array} \right.$	$\mathbf{C}$ $\mathfrak{f}_1(\mathbf{C}, \mathfrak{B}, \alpha)$ $\mathfrak{B}$	no α then the m -configuration becomes $\mathfrak{B}$.

If we were to replace $\mathbf{C}$ throughout by $\mathfrak{q}$ (say), $\mathfrak{B}$ by $\mathfrak{r}$, and α by x , we should have a complete table for the m -configuration $\mathfrak{f}(\mathfrak{q}, \mathfrak{r}, x)$. $\mathfrak{f}$ is called an “ m -configuration function” or “ m -function”.

The only expressions which are admissible for substitution in an m -function are the m -configurations and symbols of the machine. Those have to be enumerated more or less explicitly: they may include expressions such as $\mathfrak{p}(\mathfrak{e}, x)$; indeed they must if there are any m -functions used at all. If we did not insist on this explicit enumeration but simply stated that the machine had certain m -configurations (enumerated) and all m -configurations obtainable by substitution of m -configurations in certain m -functions, we should usually get an infinity of m -configurations; *e.g.*, we might say that the machine was to have the m -configuration $\mathfrak{q}$ and all m -configurations obtainable by substituting an m -configuration for $\mathbf{C}$ in $\mathfrak{p}(\mathbf{C})$. Then it would have $\mathfrak{q}$, $\mathfrak{p}(\mathfrak{q})$, $\mathfrak{p}(\mathfrak{p}(\mathfrak{q}))$, $\mathfrak{p}(\mathfrak{p}(\mathfrak{p}(\mathfrak{q})))$... as m -configurations.

Our interpretation rule then is this. We are given the names of the m -configurations of

the machine, mostly expressed in terms of m -functions. We are also given skeleton tables. All we want is the complete table for the m -configurations of the machine. This is obtained by repeated substitution in the skeleton tables.

{237} Further examples.

(In the explanations the symbol “ $\rightarrow$ ” is used to signify “the machine goes into the m -configuration ...”)

$e(\mathbf{C}, \mathfrak{B}, \alpha)$		$f(e_1(\mathbf{C}, \mathfrak{B}, \alpha), \mathfrak{B}, \alpha)$	From $e(\mathbf{C}, \mathfrak{B}, \alpha)$ the first α is erased and $\rightarrow \mathbf{C}$.
$e_1(\mathbf{C}, \mathfrak{B}, \alpha)$	E	$\mathbf{C}$	If there is no $\alpha \rightarrow \mathfrak{B}$.
$e(\mathfrak{B}, \alpha)$		$e(e(\mathfrak{B}, \alpha), \mathfrak{B}, \alpha)$	From $e(\mathfrak{B}, \alpha)$ all letters α are erased and $\rightarrow \mathfrak{B}$

The last example seems somewhat more difficult to interpret than most. Let us suppose that in the list of m -configurations of some machine there appears $e(\mathfrak{b}, x) (= \mathfrak{q}$, say). The table is

	$e(\mathfrak{b}, x)$	$e(e(\mathfrak{b}, x), \mathfrak{b}, x)$
or	$\mathfrak{q}$	$e(\mathfrak{q}, \mathfrak{b}, x)$.

Or, in greater detail:

	$\mathfrak{q}$	$e(\mathfrak{q}, \mathfrak{b}, x)$
	$e(\mathfrak{q}, \mathfrak{b}, x)$	$f(e_1(\mathfrak{q}, \mathfrak{b}, x), \mathfrak{b}, x)$
	$e_1(\mathfrak{q}, \mathfrak{b}, x)$	E
		$\mathfrak{q}$.

In this we could replace $e_1(\mathfrak{q}, \mathfrak{B}, x)$ by $\mathfrak{q}'$ and then give the table for f (with the right substitutions) and eventually reach a table in which no m -functions appeared.

$\mathfrak{p}e(\mathbf{C}, \beta)$		$f(\mathfrak{p}e_1(\mathbf{C}, \beta)\mathbf{C}, \mathfrak{a})$	From $\mathfrak{p}e(\mathbf{C}, \beta)$ the machine prints β at the end of the sequence of symbols and $\rightarrow \mathbf{C}$.
$\mathfrak{p}e_1(\mathbf{C}, \beta)$	$\left\{ \begin{array}{l} \text{Any } R, R \\ \text{None } P\beta \end{array} \right.$	$\mathfrak{p}e_1(\mathbf{C}, \beta)$	
$\mathbf{l}(\mathbf{C})$	L	$\mathbf{C}$	From $f'(\mathbf{C}, \mathfrak{B}, \alpha)$ it does the same as for $f(\mathbf{C}, \mathfrak{B}, \alpha)$ but moves to the left before $\mathbf{C}$.
$\mathbf{r}(\mathbf{C})$	R	$\mathbf{C}$	

$f'(C, \mathfrak{B}, \alpha)$	$f(l(C), \mathfrak{B}, \alpha)$	
$f''(C, \mathfrak{B}, \alpha)$	$f(r(C), \mathfrak{B}, \alpha)$	
$c(C, \mathfrak{B}, \alpha)$	$f(c_1(C), \mathfrak{B}, \alpha)$	$c(C, \mathfrak{B}, \alpha)$. The machine writes
$c_1(C)$	$pe(C, \beta)$	at the end the first symbol
β		marked α and $\rightarrow C$.

{238} The last line stands for the totality of lines obtainable from it by replacing β by any symbol which may occur on the tape of the machine concerned.

$ce(C, \mathfrak{B}, \alpha)$	$c(e(C, \mathfrak{B}, \alpha),$	$ce(\mathfrak{B}, \alpha)$. The machine
$ce(\mathfrak{B}, \alpha)$	$\mathfrak{B}, \alpha)$	copies down in order at the
	$ce(ce(\mathfrak{B}, \alpha),$	end all symbols marked α
	$\mathfrak{B}, \alpha)$	and erases the letters
		$\alpha; \rightarrow \mathfrak{B}$.
$re(C, \mathfrak{B}, \alpha, \beta)$	$f(re_1$	$re(C, \mathfrak{B}, \alpha, \beta)$. The machine
$re_1(C, \mathfrak{B}, \alpha, \beta)$	$(C, \mathfrak{B}, \alpha, \beta)$	replaces the first α by β
	$\mathfrak{B}, \alpha)$	and $\rightarrow C \rightarrow \mathfrak{B}$ if there is
	C	no α .
	$E, P\beta$	
$re(\mathfrak{B}, \alpha, \beta)$	$re(re(\mathfrak{B}, \alpha, \beta)$	$re(\mathfrak{B}, \alpha, \beta)$. The machine
	$\mathfrak{B}, \alpha, \beta)$	replaces all letters α by β ;
		$\rightarrow \mathfrak{B}$.
$cr(C, \mathfrak{B}, \alpha)$	$c(re(C, \mathfrak{B}, \alpha, a)$	$cr(\mathfrak{B}, \alpha)$ differs from ce
$cr(\mathfrak{B}, \alpha)$	$\mathfrak{B}, \alpha)$	$(\mathfrak{B}, \alpha)$ only in that the
	$cr(cr(\mathfrak{B}, \alpha),$	letters α are not erased.
	$re(\mathfrak{B}, a, \alpha), \alpha)$	The m -configuration cr
		$(\mathfrak{B}, \alpha)$ is taken up when no
		letters " a " are on the tape.

$cp(C, \mathfrak{A}, \mathfrak{E}, \alpha, \beta)$	$f'(cp_1, C_1, \mathfrak{A}, \beta), f(\mathfrak{A}, \mathfrak{E}, \beta), \alpha)$
$cp_1(C, \mathfrak{A}, \beta)$	γ
cp_2	$f'(cp_2(C, \mathfrak{A}, \beta), \mathfrak{A}, \beta)$
$(C, \mathfrak{A}, \gamma)$	$\left\{ \begin{array}{l} \gamma \\ \text{not } \gamma \end{array} \right.$
	C
	$\mathfrak{A}$.

The first symbol marked α and the first marked β are compared. If there is neither α nor $\beta \rightarrow \mathfrak{E}$. If there are both and the symbols are alike, $\rightarrow C$. Otherwise $\rightarrow \mathfrak{A}$.

$$cpe(C, \mathfrak{A}, \mathfrak{E}, \alpha, \beta) \quad cp(e(e(C, C, \beta)C, \alpha), \mathfrak{A}, \mathfrak{E}, \alpha, \beta)$$

$cpe(C, \mathfrak{A}, \mathfrak{E}, \alpha, \beta)$ differs from $cp(C, \mathfrak{A}, \mathfrak{E}, \alpha, \beta)$ in that in the case when there is

similarity the first α and β are erased.

$$\mathbf{cpe}(\mathcal{U}, \mathfrak{E}, \alpha, \beta) \quad \mathbf{cpe}(\mathbf{cpe}(\mathcal{U}, \mathfrak{E}, \alpha, \beta), \mathcal{U}, \mathfrak{E}, \alpha, \beta).$$

$\mathbf{cpe}(\mathcal{U}, \mathfrak{E}, \alpha, \beta)$. The sequence of symbols marked α is compared with the sequence marked β . $\rightarrow \mathfrak{E}$ if they are similar. Otherwise $\mathcal{U}$. Some of the symbols α and β are erased.

{239}

$$\mathbf{q}(\mathbf{C}) \begin{cases} \text{Any } R \\ \text{None } R \end{cases} \quad \begin{matrix} \mathbf{q}(\mathbf{C}) \\ \mathbf{q}_1(\mathbf{C}) \end{matrix} \quad \mathbf{q}(\mathbf{C}, \alpha). \text{ The machine finds the last symbol of form } \alpha. \rightarrow \mathbf{C}.$$

$$\mathbf{q}_1(\mathbf{C}) \begin{cases} \text{Any } R \\ \text{None } R \end{cases} \quad \begin{matrix} \mathbf{q}(\mathbf{C}) \\ \mathbf{C} \end{matrix}$$

$$\mathbf{q}(\mathbf{C}, \alpha) \quad \mathbf{q}(\mathbf{q}_1(\mathbf{C}, \alpha))$$

$$\mathbf{q}_1(\mathbf{C}, \alpha) \begin{cases} \alpha \\ \text{not } \alpha \text{ } L \end{cases} \quad \begin{matrix} \mathbf{C} \\ \mathbf{q}_1(\mathbf{C}, \alpha) \end{matrix}$$

$$\mathbf{pe}_2(\mathbf{C}, \alpha, \beta) \quad \mathbf{pe}(\mathbf{pe}(\mathbf{C}, \beta), \alpha) \quad \mathbf{pe}_2(\mathbf{pe}(\mathbf{C}, \alpha, \beta)). \text{ The machine prints } \alpha \beta \text{ at the end.}$$

$$\mathbf{ce}_2(\mathfrak{B}, \alpha, \beta) \quad \mathbf{ce}(\mathbf{ce}(\mathfrak{B}, \beta), \alpha) \quad \mathbf{ce}_3(\mathfrak{B}, \alpha, \beta, \gamma). \text{ The machine copies down at the end first the symbols marked } \alpha, \text{ then those marked } \beta, \text{ and finally those marked } \gamma; \text{ it erases the symbols } \alpha, \beta, \gamma.$$

$$\mathbf{ce}_3(\mathfrak{B}, \alpha, \beta, \gamma) \quad \mathbf{ce}(\mathbf{ce}_2(\mathfrak{B}, \beta, \gamma), \alpha)$$

$$\mathbf{e}(\mathbf{C}) \begin{cases} \mathfrak{a} \text{ } R \\ \text{Not } \mathfrak{a} \text{ } L \end{cases} \quad \begin{matrix} \mathbf{e}_1(\mathbf{C}) \\ \mathbf{e}(\mathbf{C}) \end{matrix} \quad \text{From } \mathbf{e}(\mathbf{C}) \text{ the marks are erased from all marked symbols. } \rightarrow \mathbf{C}.$$

$$\mathbf{e}_1(\mathbf{C}) \begin{cases} R, \\ \text{Any } E, \\ \text{None } R \end{cases} \quad \begin{matrix} \mathbf{e}_1(\mathbf{C}) \\ \mathbf{C} \end{matrix}$$



$$q_i \quad S_j \quad R \quad q_m$$

5. Enumeration of computable sequences.

A computable sequence γ is determined by a description of a machine which computes γ . Thus the sequence 00101101110111... is determined by the table on [p.234](#), and, in fact, any computable sequence is capable of being described in terms of such a

table.

It will be useful to put these tables into a kind of standard form. In the first place let us suppose that the table is given in the same form as the first table, for example, I on [p.233](#). That is to say, that the entry in the operations column is always of one of the forms $E : E, R : E, L : Pa : Pa, R : Pa, L : R : L$: or no entry at all. The table can always be put into this form by introducing more m -configurations. Now let us give numbers to the m -configurations, calling them $q_1, \dots, q_R$, as in [§ 1](#). The initial m -configuration is always to be called q_1 . We also give numbers to the symbols $S_1, \dots, S_m$ [{240}](#) and, in particular, blank = $S_0, 0 = S_1, 1 = S_2$. The lines of the table are now of form

<i>m-config.</i>	<i>symbol</i>	<i>operations</i>	<i>final m-config.</i>	
q_i	S_j	PS_k, L	q_m	(N_1)
q_i	S_j	PS_k, R	q_m	(N_2)
q_i	S_j	PS_k	q_m	(N_3)

Lines such as

q_i	S_j	E, R	q_m
-------	-------	--------	-------

Are to be written as

q_i	S_j	PS_0, R	q_m
-------	-------	-----------	-------

And lines such as

To be written as

q_i	S_j	PS_j, R	q_m
-------	-------	-----------	-------

In this way we reduce each line of the table to a line of one of the forms $(N_1), (N_2), (N_3)$.

From each line of form (N_1) let us form an expression $q_i S_j S_k L q_m$; from each line of form (N_2) we form an expression $q_i S_i S_k R q_m$; and from each line of form (N_3) we form an expression $q_i S_j S_k N q_m$. Let us write down all expressions so formed from the table for the machine and separate them by semi-colons. In this way we obtain a complete description of the machine. In this description we shall replace q_i by the letter “D” followed by the letter “A” repeated i times, and S_j by “D” followed by “C” repeated j times. This new description of the machine may be called the *standard description* (S.D). It is made up entirely from the letters “A”, “C”, “D”, “L”, “R”, “N”, and from “;”.

If finally we replace “A” by “1”, “C” by “2”, “D” by “3”, “L” by “4”, “R” by “5”, “N” by “6”, and “;” by “7” we shall have a description of the machine in the form of an arabic numeral. The integer represented by this numeral may be called a *description number* (D.N) of the machine. The D.N determine the S.D and the structure of the [{241}](#) machine uniquely. The machine whose D.N is n may be described as $\mathcal{M}(n)$.

To each computable sequence there corresponds at least one description number, while to no description number does there correspond more than one computable sequence. The computable sequences and numbers are therefore enumerable.

Let us find a description number for the [machine I](#) of [§3](#). When we rename the m -

configurations its table becomes:

q_1	S_0	PS_1, R	q_2
q_2	S_0	PS_0, R	q_3
q_3	S_0	PS_2, R	q_4
q_4	S_0	PS_0, R	q_1

Other tables could be obtained by adding irrelevant lines such as

q_1	S_1	PS_1, R	q_2
-------	-------	-----------	-------

Our first standard form would be

$$q_1 S_0 S_1 R q_2 ; q_2 S_0 S_0 R q_3 ; q_3 S_0 S_0 R q_4 ; \\ q_4 S_0 S_2 R q_1 ;$$

The standard description is

$$DADDCRDAA; DAADDRDAAA; \\ DAAADDCCRDA AAA; DAAAADDRDA;$$

A description number is

$$31332531173113353111731113322531111731111335317$$

and so is

$$31332531173113353111731113322531L1173111133531731323253117$$

A number which is a description number of a circle-free machine will be called a *satisfactory* number. In §8 it is shown that there can be no general process for determining whether a given number is satisfactory or not.

6. The universal computing machine.

It is possible to invent a single machine which can be used to compute any computable sequence. If this machine $\mathcal{U}$ is supplied with a tape on the beginning of which is written the S.D of some computing machine $\mathcal{M}$, {242} then $\mathcal{U}$ will compute the same sequence as $\mathcal{M}$. In this section I explain in outline the behavior of the machine. The next section is devoted to giving the complete table for $\mathcal{U}$.

Let us first suppose that we have a machine $\mathcal{M}'$ which will write down on the F -squares the successive complete configurations of $\mathcal{M}$. These might be expressed in the same form as on p.235, using the second description, (C), with all symbols on one line. Or, better, we could transform this description (as in §5) by replacing each m -configuration by “ D ” followed by “ A ” repeated the appropriate number of times, and by replacing each symbol by “ D ” followed by “ C ” repeated the appropriate number of times. The numbers of letters “ A ” and “ C ” are to agree with the numbers chosen in §5, so that, in particular, “ 0 ” is replaced by “ DC ”, “ 1 ” by “ DCC ”, and the blanks by “ D ”. These substitutions are to be made after the complete configurations have been put together, as in (C). Difficulties

arise if we do the substitution first. In each complete configuration the blanks would all have to be replaced by “D”, so that the complete configuration would not be expressed as a finite sequence of symbols.

If in the description of the machine [II](#) of [§3](#) we replace “**o**” by “DAA”, “**ə**” by “DCCC”, “**q**” by “DAAA”, then the sequence (C) becomes:

$$DA : DCCCDCCCDAAADCDDC : DCCCDCCCDAAADCDDC : \dots (C_1)$$

(This is the sequence of symbols on *F*-squares.)

It is not difficult to see that if $\mathcal{M}$ can be constructed, then so can $\mathcal{M}'$. The manner of operation of $\mathcal{M}'$ could be made to depend on having the rules of operation (*i.e.*, the S.D) of it written somewhere within itself (*i.e.* within $\mathcal{M}'$); each step could be carried out by referring to these rules. We have only to regard the rates as being capable of being taken out and exchanged or others and we have something very akin to the universal machine.

One thing is lacking: at present the machine $\mathcal{M}'$ prints no figures. We may correct this by printing between each successive pair of complete configurations the figures which appear in the new configuration but not in the old. Then (C₁) becomes

$$DDA : 0 : 0 : DCCCDCCCDAAADCDDC : DCCC\dots (C_2)$$

It is not altogether obvious that the *E*-squares leave enough room for the necessary “rough work”, but this is, in fact, the case.

The sequences of letters between the colons in expressions such as (C₁) may be used as standard descriptions of the complete configurations. When the letters are replaced by figures, as in [§5](#), we shall have a numerical {243} description of the complete configuration, which may be called its description number.



7. Detailed description of the universal machine.

A table is given below of the behaviour of this universal machine. The *m*-configurations of which the machine is capable are all those occurring in the first and last columns of the table, together with all those which occur when we write out the unabbreviated tables of those which appear in the table in the form of *m*-functions. *E.g.*, $e(\mathbf{anf})$ appears in the table and is an *m*-function. Its unabbreviated table is (see [p. 239](#))

e	$\left\{ \begin{array}{l} \mathfrak{a} \\ \text{not } \mathfrak{a} \end{array} \right.$	R	$e_1(\mathbf{anf})$
$(\mathbf{anf})$		L	$e(\mathbf{anf})$
e	$\left\{ \begin{array}{l} \text{Any} \\ \text{None} \end{array} \right.$	R, E, R	$e_1(\mathbf{anf})$
$(\mathbf{anf})$			$e(\mathbf{anf})$

Consequently $e_1(\mathbf{anf})$ is an m -configuration of $\mathcal{U}$.

When $\mathcal{U}$ is ready to start work the tape running through it bears on it the symbol $\mathfrak{a}$ on an F -square and again $\mathfrak{a}$ on the next E -square; after this, on F -squares only, comes the S.D of the machine followed by a double colon “: :” (a single symbol, on an F -square). The S.D consists of a number of instructions, separated by semi-colons.

Each instruction consists of five consecutive parts

- i) “ D ” followed by a sequence of letters “ A ”. This describes the relevant m -configuration.
- ii) “ D ” followed by a sequence of letters “ C ”. This describes the scanned symbol.
- iii) “ D ” followed by another sequence of letters “ C ”. This describes the symbol into which the scanned symbol is to be changed.
- iv) “ L ”, “ R ”, “ N ”, describing whether the machine is to move to left, right, or not at all.
- v) “ D ” followed by a sequence of letters “ A ”. This describes the final m -configuration.

The machine $\mathcal{U}$ is to be capable of printing “ A ”, “ C ”, “ D ”, “ 0 ”, “ 1 ”, “ u ”, “ v ”, “ w ”, “ x ”, “ y ”, “ z ”.

The S.D is formed from “;”, “ A ”, “ C ”, “ D ”, “ L ”, “ R ”, “ N ”.

{244} Subsidiary skeleton table.

$\mathbf{con}(\mathbf{C}, \alpha)$	$\left\{ \begin{array}{ll} \text{Not } A & R, R \\ A & L, P \alpha, R \end{array} \right.$	$\mathbf{con}(\mathbf{C}, \alpha)$	$\mathbf{con}(\mathbf{C}, \alpha)$	$\mathbf{con}(\mathbf{C}, \alpha)$. Starting from an F -square, S say, the sequence C of symbols describing a configuration closest on the right of S is marked out with letters α . $\rightarrow \mathbf{C}$.
$\mathbf{con}_1(\mathbf{C}, \alpha)$	$\left\{ \begin{array}{ll} A & R, P \alpha, R \\ D & R, P \alpha, R \end{array} \right.$	$\mathbf{con}_1(\mathbf{C}, \alpha)$	$\mathbf{con}_2(\mathbf{C}, \alpha)$	
$\mathbf{con}_1(\mathbf{C}, \alpha)$	$\left\{ \begin{array}{ll} C & R, P \alpha, R \\ \text{Not } C & R, R \end{array} \right.$	$\mathbf{con}_2(\mathbf{C}, \alpha)$	$\mathbf{C}$	$\mathbf{con}(\mathbf{C},)$. In the final configuration the machine is scanning the square which is four squares to the right of the last square of C . C is left unmarked.

The table for $\mathfrak{U}$.

$\mathfrak{b}$		$\mathfrak{f}(\mathfrak{b}_1, \mathfrak{b}_1, ::)$	$\mathfrak{b}$. The machine prints :DA on the F -squares after $:: \rightarrow \mathbf{anf}$.
$\mathfrak{b}_1$	$R, R, P :, R, R, PD, R, R, PA$	$\mathbf{anf}$	
$\mathbf{anf}$		$\mathbf{g}(\mathbf{anf}_1, :)$	$\mathbf{anf}$. The machine marks the configuration in the last complete
$\mathbf{anf}_1$		$\mathbf{con}(\mathfrak{fom}, y)$	configuration with y . $\rightarrow \mathfrak{fom}$.

fom	$\left\{ \begin{array}{l} ; \\ z \\ \text{not } z \\ \text{nor } ; \end{array} \right.$	$\begin{array}{l} R, Pz, L \\ L, L \\ L \end{array}$	con(fmp, x)	fom . The machine finds the last semi-colon not marked with z . It marks this semi-colon with z and the configuration following it with x .
fmp		cpe(e(fom, x, y), sim, x, y)		fmp . The machine compares the sequences marked x and y . It erases all letters x and y . $\rightarrow$ sim if they are alike. Otherwise $\rightarrow$ fom .

anf. Taking the long view, the last instruction relevant to the last configuration is found. It can be recognised afterwards as the instruction following the last semi-colon marked z . $\rightarrow$ **sim**.

{245}

sim		f'(sim₁, sim₁, sim)	sim . The machine marks out the instructions. That part of the instructions which refers to operations to be carried out is marked with u , and the final m -configuration with y . The letters z are erased.
sim₁		con(sim₂,)	
sim₂	$\left\{ \begin{array}{l} A \\ \text{Not } A \end{array} \right.$	$\begin{array}{l} R, Pu, R, \\ R, R \end{array}$	sim₃ sim₂
sim₃	$\left\{ \begin{array}{l} \text{Not } A \\ A \end{array} \right.$	$\begin{array}{l} L, Py \\ L, Py, R, \\ R, R \end{array}$	e(mf, z) sim₃
mf		g(mf, :)	mf . The last complete configuration is marked out into four sections. The configuration is left unmarked. The symbol directly preceding it is marked with x . The remainder of the complete configuration is divided into two parts, of which the first is marked with v and the last with w . A colon is printed after the whole. $\rightarrow$ sh .
mf₁	$\left\{ \begin{array}{l} \text{Not } A \\ A \end{array} \right.$	$\begin{array}{l} R, R \\ L, L, L, L \end{array}$	mf₁ mf₂
mf₂	$\left\{ \begin{array}{l} C \\ : \\ D \end{array} \right.$	$\begin{array}{l} R, Px, L, \\ L, L \\ R, Px, L, \\ L, L \end{array}$	mf₂ mf₄ mf₃
mf₃	$\left\{ \begin{array}{l} \text{not } : \\ : \end{array} \right.$	$\begin{array}{l} R, Pv, L, \\ L, L \end{array}$	mf₃ mf₄
mf₄		con(l(l(mf₅)),)	
mf₅	$\left\{ \begin{array}{l} \text{Any} \\ \text{None} \end{array} \right.$	$\begin{array}{l} R, Pw, R \\ P: \end{array}$	mf₅ sh

sh		$f(sh_1, inst, u)$	sh . The instructions (marked u) are
sh_1	L, L, L	sh_2	examined. If it is found that they
sh_2	$\left\{ \begin{array}{l} D \\ \text{not } D \end{array} \right.$	sh_2	involve "Print 0" or "Print 1", then
	R, R, R, R	$inst$	0: or 1: is printed at the end.
sh_3	$\left\{ \begin{array}{l} C \\ \text{not } C \end{array} \right.$	sh_4	
	R, R	$inst$	
sh_4	$\left\{ \begin{array}{l} C \\ \text{not } C \end{array} \right.$	sh_5	
	R, R	$pe_2(inst, 0, :)$	
sh_5	$\left\{ \begin{array}{l} C \\ \text{not } C \end{array} \right.$	$inst$	
		$pe_2(inst, 1, :)$	
{246} $inst$		$g(l(inst_1), u)$	$inst$. The next complete
$inst_1 \alpha$	R, E	$inst_1(\alpha)$	configuration is written down,
$inst_1(L)$		$ce_5(ov, v, y, x,$	carrying out the marked
		$u, w)$	instructions. The letters u, v, w, x, y
$inst_1(R)$		$ce_5(ov, v, y, x,$	are erased. $\rightarrow anf$.
		$u, w)$	
$inst_1(N)$		$ce_5(ov, v, y, x,$	
		$u, w)$	
ov		$e(anf)$	



8. Application of the diagonal process.

It may be thought that arguments which prove that the real numbers are not enumerable [5] would also prove that the computable numbers and sequences cannot be enumerable. It might, for instance, be thought that the limit of a sequence of computable numbers must be computable. This is clearly only true if the sequence of computable numbers is defined by some rule.

Or we might apply the diagonal process. "If the computable sequences are enumerable, let α_n be the n -th computable sequence, and let $\phi_n(m)$ be the m -th figure in α_n . Let β be the sequence with $1 - \phi_n(n)$ as its n -th figure. Since β is computable, there exists a number K such that $1 - \phi_n(n) = \phi_K(n)$ all n . Putting $n = K$, we have $1 = 2\phi_K(K)$, i.e. 1 is even. This is impossible. The computable sequences are therefore not enumerable".

The fallacy in this argument lies in the assumption that β is computable. It would be true if we could enumerate the computable sequences by finite means, but the problem of enumerating computable sequences is equivalent to the problem of finding out whether a given number is the D.N of a circle-free machine, and we have no general process for doing this in a finite number of steps. In fact, by applying the diagonal process argument correctly, we can show that there cannot be any such general process.

The simplest and most direct proof of this is by showing that, if this general process exists, then there is a machine which computes β . This proof, although perfectly sound, has the disadvantage that it may leave the reader with a feeling that “there must be something wrong”. The proof which I shall give has not this disadvantage, and gives a certain insight into the significance of the idea “circle-free”. It depends not on constructing β , but on constructing β' , whose n -th figure is $\phi_n(n)$.

{247} Let us suppose that there is such a process; that is to say, that we can invent a machine $\mathcal{D}$ which, when supplied with the S.D of any computing machine $\mathcal{M}$ will test this S.D and if $\mathcal{M}$ is circular will mark the S.D with the symbol “u” and if it is circle-free will mark it with “s”. By combining the machines $\mathcal{D}$ and $\mathcal{U}$ we could construct a machine $\mathcal{M}$ to compute the sequence β' . The machine $\mathcal{D}$ may require a tape. We may suppose that it uses the E -squares beyond all symbols on F -squares, and that when it has reached its verdict all the rough work done by $\mathcal{D}$ is erased.

The machine $\mathcal{H}$ has its motion divided into sections. In the first $N-1$ sections, among other things, the integers $1, 2, \dots, N-1$ have been written down and tested by the machine $\mathcal{D}$. A certain number, say $R(N-1)$, of them have been found to be the D.N's of circle-free machines. In the N -th section the machine $\mathcal{D}$ tests the number N . If N is satisfactory, *i.e.*, if it is the D.N of a circle-free machine, then $R(N) = 1 + R(N-1)$ and the first $R(N)$ figures of the sequence of which a D.N is N are calculated. The $R(N)$ -th figure of this sequence is written down as one of the figures of the sequence β' computed by $\mathcal{H}$. If N is not satisfactory, then $R(N) = R(N-1)$ and the machine goes on to the $(N+1)$ -th section of its motion.

From the construction of $\mathcal{H}$ we can see that $\mathcal{H}$ is circle-free. Each section of the motion of $\mathcal{H}$ comes to an end after a finite number of steps. For, by our assumption about $\mathcal{D}$, the decision as to whether N is satisfactory is reached in a finite number of steps. If N is not satisfactory, then the N -th section is finished. If N is satisfactory, this means that the machine $\mathcal{M}(N)$ whose D.N is N is circle-free, and therefore its $R(N)$ -th figure can be calculated in a finite number of steps. When this figure has been calculated and written down as the $R(N)$ -th figure of β' , the N -th section is finished. Hence $\mathcal{H}$ is circle-free.

Now let K be the D.N of $\mathcal{H}$. What does $\mathcal{H}$ do in the K -th section of its motion? It must test whether K is satisfactory, giving a verdict “s” or “u”. Since K is the D.N of $\mathcal{H}$ and since $\mathcal{H}$ is circle-free, the verdict cannot be “u”. On the other hand the verdict cannot be “s”. For if it were, then in the K -th section of its motion $\mathcal{H}$ would be bound to compute the first $R(K-1)+1 = R(K)$ figures of the sequence computed by the machine with K as its D.N and to write down the $R(K)$ -th as a figure of the sequence computed by $\mathcal{H}$. The computation of the first $R(K)-1$ figures would be carried out all right, but the

instructions for calculating the $R(K)$ -th would amount to “calculate the first $R(K)$ figures computed by H and write down the $R(K)$ -th”. This $R(K)$ -th figure would never be found. *I.e.*, $\mathcal{H}$ is circular, contrary both to what we have found in the last paragraph and to the verdict “s”. Thus both verdicts are impossible and we conclude that there can be no machine $\mathcal{D}$.

{248} We can show further that *there can be no machine $\mathcal{E}$ which, when applied with the S.D of an arbitrary machine $\mathcal{M}$, will determine whether $\mathcal{M}$ ever prints a given symbol (0 say).*

We will first show that, if there is a machine $\mathcal{E}$, then there is a general process for determining whether a given machine $\mathcal{M}$ prints 0 infinitely often. Let $\mathcal{M}_1$ be a machine which prints the same sequence as $\mathcal{M}$, except that in the position where the first 0 printed by $\mathcal{M}$ stands, $\mathcal{M}_1$ prints $\bar{0}$. $\mathcal{M}_2$ is to have the first two symbols 0 replaced by $\bar{0}$, and so on. Thus, if $\mathcal{M}$ were to print

$$A B A 0 1 A A B 0 0 1 0 A B \dots,$$

then $\mathcal{M}_1$ would print

$$A B A \bar{0} 1 A A B 0 0 1 0 A B \dots$$

and $\mathcal{M}_2$ would print

$$A B A \bar{0} 1 A A B \bar{0} 0 1 0 A B \dots$$

Now let $\mathcal{F}$ be a machine which, when supplied with the S.D of $\mathcal{M}$, will write down successively the S.D of $\mathcal{M}$, of $\mathcal{M}_1$, of $\mathcal{M}_2$, ... (there is such a machine). We combine $\mathcal{F}$ with $\mathcal{E}$ and obtain a new machine, $\mathcal{G}$. In the motion of $\mathcal{G}$ first $\mathcal{F}$ is used to write down the S.D of $\mathcal{M}$, and then $\mathcal{E}$ tests it, :0: is written if it is found that $\mathcal{M}$ never prints 0; then $\mathcal{F}$ writes the S.D of $\mathcal{M}_1$ and this is tested, :0: being printed if and only if $\mathcal{M}_1$ never prints 0; and so on. Now let us test $\mathcal{G}$ with $\mathcal{E}$. If it is found that $\mathcal{G}$ never prints 0, then $\mathcal{M}$ prints 0 infinitely often; if $\mathcal{G}$ prints 0 sometimes, then $\mathcal{M}$ does not print 0 infinitely often.

Similarly there is a general process for determining whether $\mathcal{M}$ prints 1 infinitely often. By a combination of these processes we have a process for determining whether $\mathcal{M}$ prints an infinity of figures, *i.e.* we have a process for determining whether $\mathcal{M}$ is circle-free. There can therefore be no machine $\mathcal{E}$.

The expression “there is a general process for determining ...” has been used throughout this section as equivalent to “there is a machine which will determine ...” This usage can be justified if and only if we can justify our definition of “computable”. For each of these “general process” problems can be expressed as a problem concerning a general process for determining whether a given integer n has a property $G(n)$ [*e.g.* $G(n)$ might mean “ n is satisfactory” or “ n is the Gödel representation of a provable formula”], and this is

equivalent to computing a number whose n -th figure is 1 if $G(n)$ is true and 0 if it is false. {249}

9. The extent of the computable numbers.

No attempt has yet been made to show that the “computable” numbers include all numbers which would naturally be regarded as computable. All arguments which can be given are bound to be, fundamentally, appeals to intuition, and for this reason rather unsatisfactory mathematically. The real question at issue is “What are the possible processes which can be carried out in computing a number?”

The arguments which I shall use are of three kinds.

- a. A direct appeal to intuition.
2. A proof of the equivalence of two definitions (in case the new definition has a greater intuitive appeal).
3. Giving examples of large classes of numbers which are computable.

Once it is granted that computable numbers are all “computable” several other propositions of the same character follow. In particular, it follows that, if there is a general process for determining whether a formula of the Hilbert function calculus is provable, then the determination can be carried out by a machine.

I. [Type (a)]. This argument is only an elaboration of the ideas of §1.

Computing is normally done by writing certain symbols on paper. We may suppose this paper is divided into squares like a child's arithmetic book. In elementary arithmetic the two-dimensional character of the paper is sometimes used. But such a use is always avoidable, and I think that it will be agreed that the two-dimensional character of paper is no essential of computation. I assume then that the computation is carried out on one-dimensional paper, *i.e.* on a tape divided into squares. I shall also suppose that the number of symbols which may be printed is finite. If we were to allow an infinity of symbols, then there would be symbols differing to an arbitrarily small extent. [6] The effect of this restriction of the number of symbols is not very serious. It is always possible to use sequences of symbols in the place of single symbols. Thus an Arabic numeral such as {250} 17 or 9999999999999999 is normally treated as a single symbol. Similarly in any European language words are treated as single symbols (Chinese, however, attempts to have an enumerable infinity of symbols). The differences from our point of view between the single and compound symbols is that the compound symbols, if they are too lengthy, cannot be observed at one glance. This is in accordance with experience. We cannot tell at a glance whether 9999999999999999 and 9999999999999999 are the same.

The behaviour of the computer at any moment is determined by the symbols which he is observing. and his “state of mind” at that moment. We may suppose that there is a bound B to the number of symbols or squares which the computer can observe at one moment. If he wishes to observe more, he must use successive observations. We will also suppose that the number of states of mind which need be taken into account is finite. The reasons for this are of the same character as those which restrict the number of symbols. If we admitted an infinity of states of mind, some of them will be “arbitrarily close” and will be confused. Again, the restriction is not one which seriously affects computation, since the use of more complicated states of mind can be avoided by writing more symbols on the tape.

Let us imagine the operations performed by the computer to be split up into “simple operations” which are so elementary that it is not easy to imagine them further divided. Every such operation consists of some change of the physical system consisting of the computer and his tape. We know the state of the system if we know the sequence of symbols on the tape, which of these are observed by the computer (possibly with a special order), and the state of mind of the computer. We may suppose that in a simple operation not more than one symbol is altered. Any other changes can be set up into simple changes of this kind. The situation in regard to the squares whose symbols may be altered in this way is the same as in regard to the observed squares. We may, therefore, without loss of generality, assume that the squares whose symbols are changed are always “observed” squares.

Besides these changes of symbols, the simple operations must include changes of distribution of observed squares. The new observed squares must be immediately recognisable by the computer. I think it is reasonable to suppose that they can only be squares whose distance from the closest of the immediately previously observed squares does not exceed a certain fixed amount. Let us say that each of the new observed squares is within L squares of an immediately previously observed square. In connection with “immediate recognisability”, it may be thought that there are other kinds of square which are immediately recognisable. In particular, squares marked by special symbols might be taken as immediately recognisable. Now if these squares are marked only by single symbols there can be only a finite number of them, and we should not upset our theory by adjoining these marked squares to the observed squares. If, on the other hand, they are marked by a sequence of symbols, we cannot regard the process of recognition as a simple process. This is a fundamental point and should be illustrated. In most mathematical papers the equations and theorems are numbered. Normally the numbers do not go beyond (say) 1000. It is, therefore, possible to recognise a theorem at a glance by its number. But if the paper was very long, we might reach Theorem 157767733443477; then, farther on in the paper, we might find “... hence (applying Theorem 157767733443477) we have...”. In order to make sure which was the relevant theorem we should have to compare the two numbers figure by figure, possibly ticking the figures off in pencil to make sure of their not being counted twice. If in spite of this it is still thought that there are other “immediately recognisable” squares, it does not upset my contention so long as these squares can be found by some process of which my type of machine is capable. This idea is developed in [III](#) below.

The simple operations must therefore include:

- (a) Changes of the symbol on one of the observed squares.
- (b) Changes of one of the squares observed to another square within L squares of one of the previously observed squares.

It may be that some of these changes necessarily involve a change of state of mind. The most general single operation must therefore be taken to be one of the following:

- A. A possible change (a) of symbol together with a possible change of state of mind.
- B. A possible change (b) of observed squares, together with a possible change of state of mind.

The operation actually performed is determined, as has been suggested on [p.250](#), by the state of mind of the computer and the observed symbols. In particular, they determine the state of mind of the computer after the operation is carried out.

We may now construct a machine to do the work of this computer. To each state of mind of the computer corresponds an “ m -configuration” of the machine. The machine scans B squares corresponding to the B squares observed by the computer. In any move the machine can change a symbol on a scanned square or can change anyone of the scanned squares to another square distant not more than L squares from one of the other scanned {252} squares. The move which is done, and the succeeding configuration, are determined by the scanned symbol and the m -configuration. The machines just described do not differ very essentially from computing machines as defined in §2, and corresponding to any machine of this type a computing machine can be constructed to compute the same sequence, that is to say the sequence computed by the computer.

II. [Type (b)].

If the notation of the Hilbert functional calculus [7] is modified so as to be systematic, and so as to involve only a finite number of symbols, it becomes possible to construct an automatic [8] machine $\mathcal{K}$ which will find all the provable formulae of the calculus.[9]

Now let α be a sequence, and let us denote by $G_\alpha(x)$ the proposition “The x -th figure of α is 1”, so that [10] $\neg G_\alpha(x)$ means “The x -th figure of α is 0”. Suppose further that we can find a set of properties which define the sequence α and which can be expressed in terms of $G_\alpha(x)$ and of the propositional functions $N(x)$ meaning “ x is a non-negative integer” and $F(x, y)$ meaning “ $y = x + 1$ ”. When we join all these formulae together conjunctively we shall have a formula, $\mathfrak{A}$ say, which defines α . The terms of $\mathfrak{A}$ must include the necessary parts of the Peano axioms, viz.,

$$(\exists u) N(u) \ \& \ (x) (N(x) \rightarrow (\exists y) F(x, y)) \ \& \ (F(x, y) \rightarrow N(y)),$$

which we will abbreviate to P .

When we say “ $\mathfrak{A}$ defines α ”, we mean that $\neg \mathfrak{A}$ is not a provable formula, and also that, for each n , one of the following formulae (A_n) or (B_n) is provable.

$$\begin{aligned} \mathfrak{A} \ \& \ F^{(n)} \rightarrow G\Box(u^{(n)}), & (A_n)[11] \\ \mathfrak{A} \ \& \ F^{(n)} \rightarrow (\neg G\Box(u^{(n)})), & (B_n) \end{aligned}$$

where $F^{(n)}$ stands for $F(u, u') \ \& \ F(u', u'') \ \& \ \dots \ F(u^{(n-1)}, u^{(n)})$.

{253} I say that α is then a computable sequence: a machine $\mathcal{K}\Box$ to compute α can be obtained by a fairly simple modification of $\mathcal{K}$.

We divide the motion of $\mathcal{K}\Box$ into sections. The n -th section is devoted to finding the n -th figure of α . After the $(n - 1)$ -th section is finished a double colon :: is printed after all the symbols, and the succeeding work is done wholly on the squares to the right of this double colon. The first step is to write the letter “A” followed by the formula (A_n) and then “B” followed by (B_n) . The machine $\mathcal{K}\Box$ then starts to do the work of $\mathcal{K}$, but whenever a provable formula is found, this formula is compared with (A_n) and with (B_n) . If it is the same formula as (A_n) , then the figure “1” is printed, and the n -th section

is finished. If it is (B_n) , then “0” is printed and the section is finished. If it is different from both, then the work of $\mathcal{K}$ is continued from the point at which it had been abandoned. Sooner or later one of the formulae (A_n) or (B_n) is reached; this follows from our hypotheses about α and $\mathfrak{A}$, and the known nature of $\mathcal{K}$. Hence the n -th section will eventually be finished; $\mathcal{K}_\alpha$ is circle-free; α is computable.

It can also be shown that the numbers α definable in this way by the use of axioms include all the computable numbers. This is done by describing computing machines in terms of the function calculus.

It must be remembered that we have attached rather a special meaning to the phrase “ $\mathfrak{A}$ defines α ”. The computable numbers do not include all (in the ordinary sense) definable numbers. Let δ be a sequence whose n -th figure is 1 or 0 according as n is or is not satisfactory. It is an immediate consequence of the theorem of §8 that δ is not computable. It is (so far as we know at present) possible that any assigned number of figures of δ can be calculated, but not by a uniform process. When sufficiently many figures of δ have been calculated, an essentially new method is necessary in order to obtain more figures.

III. This may be regarded as a modification of I or as a corollary of II.

We suppose, as in I, that the computation is carried out on a tape; but we avoid introducing the “state of mind” by considering a more physical and definite counterpart of it. It is always possible for the computer to break off from his work, to go away and forget all about it, and later to come back and go on with it. If he does this he must leave a note of instructions (written in some standard form) explaining how the work is to be continued. This note is the counterpart of the “state of mind”. We will suppose that the computer works by such a desultory manner that he never does more than one step at a sitting. The note of instructions must enable him to carry out one step and write the next note. Thus the state of progress of the computation at any stage is completely determined by the note of {254} instructions and the symbols on the tape. That is, the state of the system may be described by a single expression (sequence of symbols), consisting of the symbols on the tape followed by Δ (which we suppose not to appear elsewhere) and then by the note of instructions. This expression may be called the “state formula”. We know that the state formula at any given stage is determined by the state formula before the last step was made, and we assume that the relation of these two formulae is expressible in the functional calculus. In other words we assume that there is an axiom $\mathfrak{A}$ which expresses the rules governing the behaviour of the computer, in terms of the relation of the state formula at any stage to the state formula at the proceeding stage. If this is so, we can construct a machine to write down the successive state formulae, and hence to compute the required number.



10. Examples of large classes of numbers which are computable.

It will be useful to begin with definitions of a computable function of an integral variable and of a computable variable, etc. There are many equivalent ways of defining a computable function of an integral variable. The simplest is, possibly, as follows. If γ is a computable sequence in which 0 appears infinitely [12] often, and n is an integer, then let us define $\xi(\gamma, n)$ to be the number of figures 1 between the n -th and the $(n+1)$ -th

figure 0 in γ . Then $\phi(n)$ is computable if, for all n and some γ , $\phi(n) = \xi(\gamma, n)$. An equivalent definition is this. Let $H(x, y)$ mean $\phi(x) = y$. Then if we can find a contradiction-free axiom $\mathfrak{A}_\phi$ such that $\mathfrak{A}_\phi \rightarrow P$, and if for each integer n there exists an integer N , such that

$$\mathfrak{A}_\phi \ \& \ F^{(N)} \rightarrow H(u^{(n)}, u^{(\phi(n))})$$

and such that, if $m \neq \phi(n)$, then, for some N' ,

$$\mathfrak{A}_\phi \ \& \ F^{(N')} \rightarrow (-H(u^{(n)}, m)),$$

then ϕ may be said to be a computable function.

We cannot define general computable functions of a real variable, since there is no general method of describing a real number, but we can define a computable function of a computable variable. If n is satisfactory, let γ_n be the number computed by $\mathcal{M}(n)$, and let

$$\alpha_n = \tan(\pi(\gamma_n - 1/2)),$$

{255} unless $\gamma_n = 0$ or $\gamma_n = 1$, in either of which cases $\alpha_n = 0$. Then, as n runs through the satisfactory numbers, α_n runs through the computable numbers. [13] Now let $\phi(n)$ be a computable function which can be shown to be such that for any satisfactory argument its value is satisfactory. [14] Then the function f , defined by $f(\alpha_n) = \alpha_{\phi(n)}$, is a computable function and all computable functions of a computable variable are expressible in this form.

Similar definitions may be given of computable functions of several variables, computable-valued functions of an integral variable, etc.

I shall enunciate a number of theorems about computability, but I shall prove only (ii) and a theorem similar to (iii).

i) A computable function of a computable function of an integral or computable variable is computable.

ii) Any function of an integral variable defined recursively in terms of computable functions is computable. I.e. if $\phi(m, n)$ is computable, and r is some integer, then $\eta(n)$ is computable, where

$$\eta(0) = r,$$

$$\eta(n) = \phi(n, \eta(n-1)).$$

iii) If $\phi(m, n)$ is a computable function of two integral variables, then $\phi(n, n)$ is a computable function of n .

iv) If $\phi(n)$ is a computable function whose value is always 0 or 1, then the sequence whose n -th figure is $\phi(n)$ is computable. Dedekind's theorem does not hold in the ordinary form if we replace "real" throughout by 'computable'. But it holds in the following form:

v) If $G(\alpha)$ is a propositional function of the computable numbers and

$$a) (\exists \alpha)(\exists \beta) \{ G(\alpha) \& (- G(\beta)) \},$$

$$b) G(\alpha) \& (- G(\beta)) \rightarrow (\alpha < \beta),$$

and there is a general process for determining the truth value of $G(\alpha)$, then {256} there is a computable number ξ such that

$$\begin{aligned} G(\alpha) &\rightarrow \alpha \leq \xi, \\ -G(\alpha) &\rightarrow \alpha \geq \xi. \end{aligned}$$

In other words, the theorem holds for any section of the computables such that there is a general process for determining to which class a given number belongs.

Owing to this restriction of Dedekind's theorem, we cannot say that a computable bounded increasing sequence of computable numbers has a computable limit. This may possibly be understood by considering a sequence such as

$$-1, -1/2, -1/4, -1/8, -1/16, 1/2, \dots$$

On the other hand, (v) enables us to prove

vi) If α and β are computable and $\alpha < \beta$ and $\phi(\alpha) < 0 < \phi(\beta)$, where $\phi(\alpha)$ is a computable increasing continuous function, then there is a unique computable number γ , satisfying $\alpha < \gamma < \beta$ and $\phi(\gamma) = 0$.

Computable convergence.

We shall say that a sequence β_n of computable numbers *converges computably* if there is a computable integral valued function $N(\varepsilon)$ of the computable variable ε , such that we can show that, if $\varepsilon > 0$ and $n > N(\varepsilon)$ and $m > N(\varepsilon)$, then $|\beta_n - \beta_m| < \varepsilon$.

We can then show that

vii) A power series whose coefficients form a computable sequence of computable numbers is computably convergent at all computable points in the interior of its interval of convergence.

viii) The limit of a computably convergent sequence is computable.

And with the obvious definition of “uniformly computably convergent”:

ix) The limit of a uniformly computably convergent computable sequence of computable functions is a computable function. Hence

x) The sum of a power series whose coefficients form a computable sequence is a computable function in the interior of its interval of convergence.

From (viii) and $\pi = 4(1 - 1/3 + 1/5 - \dots)$ we deduce that π is computable. From $e = 1 + 1 + 1/2!$

$+ \frac{1}{3!} \dots$ we deduce that e is computable.

{257} From (vi) we deduce that all real algebraic numbers are computable.

From (vi) and (x) we deduce that the real zeros of the Bessel functions are computable.

Proof of (ii).

Let $H(x, y)$ mean “ $\eta(x)=y$ ”, and let $K(x, y, z)$ mean “ $\phi(x, y)=z$ ”. $\mathfrak{A}_\phi$ is the axiom for $\phi(x, y)$. We take $\mathfrak{A}_\eta$ to be

$$\begin{aligned} \mathfrak{A}_\phi &\& P \& (F(x, y) \rightarrow G(x, y)) \& (G(x, y) \& G(y, z) \rightarrow G(x, z)) \\ &\& (F^{(r)} \rightarrow H(u, u^{(r)})) \& (F(v, w) \& H(v, x) \& K(w, x, z) \rightarrow H \\ &\quad (w, z)) \\ &\& [H(w, z) \& G(z, t) \vee G(t, z) \rightarrow (\neg H(w, t))]. \end{aligned}$$

I shall not give the proof of consistency of $\mathfrak{A}_\eta$. Such a proof may be constructed by the methods used in Hilbert and Bernays, *Grundlagen der Mathematik* (Berlin, 1934), p.209 *et seq.* The consistency is also clear from the meaning.

Suppose that for some n, N , we have shown

$$\mathfrak{A}_\phi \& F^{(N)} \rightarrow H(u^{(n-1)}, u^{(\eta(n-1))},$$

then, for some M ,

$$\begin{aligned} \mathfrak{A}_\eta \& F^{(M)} &\rightarrow K(u^{(n)}, u^{(\eta(n-1))}, u^{(\eta(n))}) \\ \mathfrak{A}_\eta \& F^{(M)} &\rightarrow F(u^{(n-1)}, u^{(n)} \& H(u^{(n-1)}, u^{(\eta(n-1))}) \\ \text{—————} &\& Ku^{(n)}, u^{(\eta(n-1))}, u^{(\eta(n))}) \end{aligned}$$

and

$$\begin{aligned} \mathfrak{A}_\eta \& F^{(M)} &\rightarrow [F(u^{(n-1)}, u^{(n)} \& H(u^{(n-1)}, u^{(\eta(n-1))}) \\ \text{—————} &\& Ku^{(n)}, u^{(\eta(n-1))}, u^{(\eta(n))}) \rightarrow H(u^{(n)}, u^{(\eta(n))})]. \end{aligned}$$

Hence $\mathfrak{A}_\eta \& F^{(M)} \rightarrow H(u^{(n)}, u^{(\eta(n))})$.

Also $\mathfrak{A}_\eta \& F^{(r)} \rightarrow H(u, u^{(\eta(0))})$.

Hence for each n some formula of the form

$$\mathfrak{A}_\eta \& F^{(M)} \rightarrow H(u^{(n)}, u^{(\eta(n))})$$

is provable. Also, if $M' \geq M$ and $M' \geq m$ and $m \neq \eta(u)$, then

$$\mathfrak{A}_\eta \& F^{(M')} \rightarrow G(u^{(\eta^{(n)})}, u^{(m)}) \vee G(u^{(m)}, u^{(\eta^{(n)})})$$

{258} and

$$\mathfrak{A}_\eta \& F^{(M')} \rightarrow [\{G(u^{(\eta^{(n)})}, u^{(m)}) \vee G(u^{(m)}, u^{(\eta^{(n)})}) \& H(u^{(n)}, u^{(\eta^{(n)})})\} \rightarrow (-H(u^{(n)}, u^{(m)}))].$$

$$\text{Hence } \mathfrak{A}_\eta \& F^{(M')} \rightarrow (-H(u^{(n)}, u^{(m)}))$$

The conditions of our second definition of a computable function are therefore satisfied. Consequently η is a computable function.

Proof of a modified form of (iii).

Suppose that we are given a machine $\mathcal{N}$, which, starting with a tape bearing on it $\mathfrak{a}\mathfrak{a}$ followed by a sequence of any number of letters “ F ” on F -squares and in the m -configuration b , will compute a sequence γ_n depending on the number n of letters “ F ”. If $\phi_n(m)$ is the m -th figure of γ_n , then the sequence β whose n -th figure is $\phi_n(n)$ is computable.

We suppose that the table for $\mathcal{N}$ has been written out in such a way that in each line only one operation appears in the operations column. We also suppose that Ξ , Θ , $\bar{0}$ and $\bar{1}$ do not occur in the table, and we replace $\mathfrak{a}$ throughout by Ξ , 0 by $\bar{0}$ and 1 by $\bar{1}$. Further substitutions are then made. Any line of form

$$\mathfrak{A} \quad \alpha \quad P\bar{0} \quad \mathfrak{B}$$

we replace by

$$\mathfrak{A} \quad \alpha \quad P\bar{0} \quad \mathbf{re}(\mathfrak{B}, \mathfrak{u}, h, k)$$

and any line of the form

$$\mathfrak{A} \quad \alpha \quad P\bar{1} \quad \mathfrak{B}$$

$$\text{by } \mathfrak{A} \quad \alpha \quad P\bar{1} \quad \mathbf{re}(\mathfrak{B}, \mathfrak{v}, h, k)$$

and we add to the table the following lines:

$$\begin{array}{ll} \mathfrak{u} & \mathbf{pe}(\mathfrak{u}_1, 0) \\ \mathfrak{u}_1 & R, Pk, R, P\Theta, R, P\Theta \quad \mathfrak{u}_2 \\ \mathfrak{u}_2 & \mathbf{re}(\mathfrak{u}_3, \mathfrak{u}_3, k, h) \\ \mathfrak{u}_2 & \mathbf{pe}(\mathfrak{u}_2, F) \end{array}$$

and similar lines with $\mathfrak{v}$ for $\mathfrak{u}$ and 1 for 0 together with the following line

$\mathfrak{c} \quad R, P\Xi, R, Ph \quad \mathfrak{b}.$



We then have the table for the machine $\mathcal{N}'$ which computes β . The initial m -configuration is $\mathfrak{c}$, and the initial scanned symbol is the second $\mathfrak{a}$.

{259}

11. Application to the Entscheidungsproblem.

The results of §8 have some important applications. In particular, they can be used to show that the Hilbert Entscheidungsproblem can have no solution. For the present I shall confine myself to proving this particular theorem. For the formulation of this problem I must refer the reader to Hilbert and Ackermann's *Grundzüge der Theoretischen Logik* (Berlin, 1931), chapter 3.

I propose, therefore, to show that there can be no general process for determining whether a given formula $\mathfrak{A}$ of the functional calculus $\mathbf{K}$ is provable, i.e. that there can be no machine which, supplied with any one $\mathfrak{A}$ of these formulae, will eventually say whether $\mathfrak{A}$ is provable.

It should perhaps be remarked what I shall prove is quite different from the well-known results of Gödel [15]. Gödel has shown that (in the formalism of Principia Mathematica) there are propositions $\mathfrak{A}$ such that neither $\mathfrak{A}$ nor $\neg\mathfrak{A}$ is provable. As a consequence of this, it is shown that no proof of consistency of Principia Mathematica (or of $\mathbf{K}$) can be given within that formalism. On the other hand, I shall show that there is no general method which tells whether a given formula $\mathfrak{A}$ is provable in $\mathbf{K}$, or, what comes to the same, whether the system consisting of $\mathbf{K}$ with $\neg\mathfrak{A}$ adjoined as an extra axiom is consistent.

If the negation of what Gödel has shown had been proved, i.e. if, for each $\mathfrak{A}$, either $\mathfrak{A}$ or $\neg\mathfrak{A}$ is provable, then we should have an immediate solution of the Entscheidungsproblem. For we can invent a machine $\mathcal{K}$ which will prove consecutively all provable formulae. Sooner or later $\mathcal{K}$ will reach either $\mathfrak{A}$ or $\neg\mathfrak{A}$. If it reaches $\mathfrak{A}$, then we know that $\mathfrak{A}$ is provable. If it reaches $\neg\mathfrak{A}$, then, since $\mathbf{K}$ is consistent (Hilbert and Ackermann, p.65), we know that $\mathfrak{A}$ is not provable.

Owing to the absence of integers in $\mathbf{K}$ the proofs appear somewhat lengthy. The underlying ideas are quite straightforward.

Corresponding to each computing machine $\mathcal{M}$ we construct a formula $\text{Un}(\mathcal{M})$ and we show that, if there is a general method for determining whether $\text{Un}(\mathcal{M})$ is provable, then there is a general method for determining whether $\mathcal{M}$ ever prints 0.

The interpretations of the propositional functions involved are as follows:

$R_{S_1}(x, y)$ is to be interpreted as "in the complete configuration x (of $\mathcal{M}$) the symbol on the square y is S ".

{260} $I(x, y)$ is to be interpreted as “in the complete configuration x the square y is scanned”.

$K_{q_m}(x)$ is to be interpreted as “in the complete configuration x the m -configuration is q_m ”.

$F(x, y)$ is to be interpreted as “ y is the immediate successor of x ”.

$\text{Inst}\{q_i S_j S_k L_{q_l}\}$ is to be an abbreviation for

$$(x, y, x', y') \left\{ (R_{S_j}(x, y) \& I(x, y) \& K_{q_i}(x) \& F(x, x') \& F(y', y)) \right. \\ \rightarrow (I(x', y') \& R_{S_k}(x', y) \& K_{q_l}(x') \\ \left. \& (z) [F(y', z) \vee (R_{S_j}(x', z) \rightarrow R_{S_k}(x', z))]) \right\}.$$

$\text{Inst}\{q_i, S_j, S_k, R_{q_l}\}$ and $\text{Inst}\{q_i, S_j, S_k, N_{q_l}\}$

are to be abbreviations for other similarly constructed expressions.

Let us put the description of $\mathcal{M}$ into the first standard form of §6. This description consists of a number of expressions such as “ q_i, S_j, S_k, L_{q_l} ” (or with R or N substituted for L). Let us form all the corresponding expressions such as $\text{Inst}\{q_i, S_j, S_k, L_{q_l}\}$ and take their logical sum. This we call $\text{Des}(\mathcal{M})$.

The formula $\text{Un}(\mathcal{M})$ is to be

$$(\exists u) [N(u) \& (x) (N(x) \rightarrow \exists x' F(x, x')) \\ \& (y, z) (F(y, z) \rightarrow N(y) \& N(z)) \& (y) R_{S_0}(u, y) \\ \& I(u, u) \& K_{q_1}(u) \& \text{Des}(\mathcal{M})] \\ \rightarrow (\exists s) (\exists t) [N(s) \& N(t) \& R_{S_1}(s, t)].$$

$[N(u) \& \dots \text{Des}(\mathcal{M})]$ may be abbreviated to $A(\mathcal{M})$.

When we substitute the meanings suggested on p.259 – 60 we find that $\text{Un}(\mathcal{M})$ has the interpretation “in some complete configuration of $\mathcal{M}$, S_1 (i.e. 0) appears on the tape”. Corresponding to this I prove that

a) If S_1 appears on the tape in some complete configuration of $\mathcal{M}$, then $\text{Un}(\mathcal{M})$ is provable.

b) If $\text{Un}(\mathcal{M})$ is provable, then S_1 appears on the tape in some complete configuration of $\mathcal{M}$.

When this has been done, the remainder of the theorem is trivial.

{261} LEMMA1. *If S_1 appears on the tape in some complete configuration of $\mathcal{M}$, then $\text{Un}(\mathcal{M})$ is provable.*

We have to show how to prove $\text{Un}(\mathcal{M})$. Let us suppose that in the n -th complete configuration the sequence of symbols on the tape is $S_{r(n,0)}, S_{r(n,1)}, \dots, S_{r(n,n)}$, followed by nothing but blanks, and that the scanned symbol is the $i(n)$ -th, and that the m -configuration is $q_{k(n)}$. Then we may form the proposition

$$\begin{aligned} & R_{S_{r(n,0)}}(u^{(n)}, u) \& R_{S_{r(n,1)}}(u^{(n)}, u') \& \dots \& R_{S_{r(n,n)}}(u^{(n)}, u^{(n)}) \\ & \& I(u^{(n)}, u^{(i(n))}) \& K_{q_{k(n)}}(u^{(n)}) \\ & \& (y) F((y, u') \vee F(u, y) \vee F(u', y) \vee \dots \vee F(u^{(n-1)}, y) \vee R_{S_0}(u^{(n)}, y)) \end{aligned}$$

which we may abbreviate to CC_n .

As before, $F(u, u') \& F(u, u'') \& \dots \& F(u^{(r-1)}, u^{(r)})$, is abbreviated to $F^{(r)}$.

I shall show that all formulae of the form $A(\mathcal{M}) \& F^{(n)} \rightarrow CC_n$ (abbreviated to CF_n) are provable. The meaning of CF_n is “The n -th complete configuration of $\mathcal{M}$ is so and so”, where “so and so” stands for the actual n -th complete configuration of $\mathcal{M}$. That CF_n should be provable is therefore to be expected.

CF_0 is certainly provable, for in the complete configuration the symbols are all blanks, the m -configuration is q_1 , and the scanned square is u , i.e. CC_0 is

$$(y)R_{S_0}(u, y) \& I(u, u) \& K_{q_1}(u).$$

$A(\mathcal{M}) \rightarrow CC_0$ is then trivial.

We next show that $CF_n \rightarrow CF_{n+1}$ is provable for each n . There are three cases to consider, according as in the move from the n -th to the $(n+1)$ -th configuration the machine moves to left or to right or remains stationary. We suppose that the first case applies, i.e. the machine moves to the left. A similar argument applies in the other cases. If $r(n, i(n))=a$, $r(n+1, i(n+1))=c$, $k(i(n))=b$, and $k(i(n+1))=d$, then $\text{Des}(\mathcal{M})$ must include $\text{Inst}\{q_a S_b S_d Lq_c\}$ as one of its terms, i.e.

$$\text{Des}(\mathcal{M}) \rightarrow \text{Inst}\{q_a S_b S_d Lq_c\}.$$

Hence $A(\mathcal{M}) \& F^{(n+1)} \rightarrow \text{Inst}\{q_a S_b S_d Lq_c\} \& F^{(n+1)}$.

But $\text{Inst}\{qa\ Sb\ Sd\ Lqc\} \ \&\ F^{(n+1)} \rightarrow (CC_n \rightarrow CC_{n+1})$

is provable, and so therefore is

$$A(\mathcal{M}) \ \&\ F^{(n+1)} \rightarrow (CC_n \rightarrow CC_{n+1})$$

{262} and $A(\mathcal{M}) \ \&\ F^{(n)} \rightarrow CC_n \rightarrow (A(\mathcal{M}) \ \&\ F^{(n+1)} \rightarrow CC_{n+1})$

$$\text{i.e. } CF_n \rightarrow CF_{n+1}.$$

CF_n is provable for each n . Now it is the assumption of this lemma that S_1 appears somewhere, in some complete configuration, in the sequence of symbols printed by $\mathcal{M}$; that is, for some integers N, K , CC_N has $R_{S_1}(u^{(N)}, u^{(K)})$ as one of its terms, and therefore $CC_N \rightarrow R_{S_1}(u^{(N)}, u^{(K)})$ is provable. We have then

$$CC_N \rightarrow R_{S_1}(u^{(N)}, u^{(K)})$$

and $A(\mathcal{M}) \ \&\ F^{(n)} \rightarrow CC^N$

We also have $(\exists u) A(\mathcal{M}) \rightarrow (\exists u) (\exists u') \dots (\exists u^{(N')}) A(\mathcal{M}) \ \&\ F^{(N')}$,

where $N' = \max(N, K)$. And so

$$(\exists u) A(\mathcal{M}) \rightarrow (\exists u) (\exists u') \dots (\exists u^{(N')}) R_{S_1}(u^{(N)}, u^{(K)}),$$

$$(\exists u) A(\mathcal{M}) \rightarrow (\exists u) (\exists u^{(N)}) (\exists u^{(K)}) (\exists u^{(N)}, u^{(K)}),$$

$$(\exists u) A(\mathcal{M}) \rightarrow (\exists s) (\exists t) R_{S_1}(s, t),$$

i.e. $\text{Un}(\mathcal{M})$ is provable.

This completes the proof of Lemma 1.

LEMMA 2. *If $\text{Un}(\mathcal{M})$ is provable, then S_1 appears on the tape in so-complete configuration of $\mathcal{M}$.*

If we substitute any propositional functions for function variables in a provable formula, we obtain a true proposition. In particular, if we substitute the meanings tabulated on [pp. 259 – 260](#) in $\text{Un}(\mathcal{M})$, we obtain a true proposition with the meaning “ S_1 appears somewhere on the tape in some complete configuration of $\mathcal{M}$ ”.

We are now in a position to show that the Entscheidungsproblem cannot be solved. Let us suppose the contrary. Then there is a general (mechanical) process for determining whether $\text{Un}(\mathcal{M})$ is provable. By Lemmas 1 and 2, this implies that there is a process for determining whether $\mathcal{M}$ ever prints 0, and this is impossible, by [§8](#). Hence the Entscheidungsproblem cannot be solved.

In view of the large number of particular cases of solutions of the Entscheidungsproblem for formulae with restricted systems of quantors, it {263} is interesting to express $\text{Un}(\mathcal{M})$ in a form in which all quantors are at the beginning. $\text{Un}(\mathcal{M})$ is, in fact, expressible in the form

$$(u)(\exists x)(w)(\exists u_1) \dots (\exists u_n) \mathfrak{B}, \quad (\text{I})$$

where $\mathfrak{B}$ contains no quantors, and $n = 6$. By unimportant modifications we can obtain a formula, with all essential properties of $\text{Un}(\mathcal{M})$, which is of form (I) with $n = 5$.



Added 28 August, 1936. **APPENDIX.**

Computability and effective calculability

The theorem that all effectively calculable (λ -definable) sequences are computable and its converse are proved below in outline. It is assumed that the terms “well-formed formula” (W.F.F.) and “conversion” as used by Church and Kleene are understood. In the second of these proofs the existence of several formulae is assumed without proof; these formulae may be constructed straightforwardly with the help of, *e.g.*, the results of Kleene in “A theory of positive integers in formal logic”, *American Journal of Math.*, 57 (1935), 153-173, 219-244.

The W.F.F. representing an integer n will be denoted by N_n . We shall say that a sequence γ whose n -th figure is $\phi_\gamma(n)$ is λ -definable or effectively calculable if $1 + \phi_\gamma(u)$ is a λ -definable function of n , *i.e.* if there is a W.F.F. M_γ such that, for all integers n ,

$$\{M_\gamma\}(N_n) \text{ conv } N_{\phi_\gamma(n)+1},$$

i.e. $\{M_\gamma\}(N_n)$ is convertible into $\lambda xy.x(x(y))$ or into $\lambda xy.x(y)$ according as the n -th figure of λ is 1 or 0.

To show that every λ -definable sequence γ is computable, we have to show how to construct a machine to compute γ . For use with machines it is convenient to make a trivial modification in the calculus of conversion. This alteration consists in using x , x' , x'' , ... as variables instead of a , b , c , We now construct a machine $\mathcal{L}$ which, when supplied with the formula M_γ , writes down the sequence γ . The construction of $\mathcal{L}$ is somewhat similar to that of the machine $\mathcal{K}$ which proves all provable formulae of the functional calculus. We first construct a choice machine $\mathcal{L}_1$ which, if supplied with a W.F.F., M say, and suitably manipulated, obtains any formula into which M is convertible. $\mathcal{L}_1$ can then be modified so as to yield an automatic machine $\mathcal{L}_2$ which obtains successively all the formulae {264} into which M is convertible ([cf- foot-note p.252](#)). The machine $\mathcal{L}$ includes $\mathcal{L}_2$ as a part. The motion of the machine $\mathcal{L}$ when supplied with the formula M_γ is divided into sections of which the n -th is devoted to finding the n -th figure of γ . The first stage in this n -th section is the formation of $\{M_\gamma\}(N_n)$. This formula is then supplied to the machine $\mathcal{L}_2$, which converts it successively

into various other formulae. Each formula into which it is convertible eventually appears, and each, as it is found, is compared with

$$\lambda x \left[\lambda' x \left[\{x\}(\{x\}(x')) \right] \right], \text{ i.e. } N_2,$$

and with $\lambda x \left[\lambda x' [\{x\}(x')] \right], \text{ i.e. } N_1.$

If it is identical with the first of these, then the machine prints the figure 1 and the n -th section is finished. If it is identical with the second, then 0 is printed and the section is finished. If it is different from both, then the work of $\mathcal{L}_2$ is resumed. By hypothesis, $\{M_\gamma\}(N_n)$ is convertible into one of the formulae N_2 or N_1 ; consequently the n -th section will eventually be finished, i.e. the n -th figure of γ will eventually be written down.

To prove that every computable sequence γ is λ -definable, we must show how to and a formula M_γ such that, for all integers n ,

$$\{M_\gamma\}(N_n) \text{ conv } N_{1+\phi_\gamma(n)}.$$

Let $\mathcal{M}$ be a machine which computes γ and let us take some description of the complete configurations of $\mathcal{M}$ by means of numbers, e.g. we may take the D.N of the complete configuration as described in §6. Let $\xi(n)$ be the D.N of the n -th complete configuration of $\mathcal{M}$. The table for the machine $\mathcal{M}$ gives us a relation between $\xi(n+1)$ and $\xi(n)$ of the form

$$\xi(n+1) = p_\gamma(\xi(n)),$$

where p_γ is a function of very restricted, although not usually very simple, form: it is determined by the table for $\mathcal{M}$. p_γ is λ -definable (I omit the proof of this), i.e. there is a W.F.F. A_γ such that, for all integers n ,

$$\{A_\gamma\}(N_{\xi(n)}) \text{ conv } N_{\xi(n+1)}.$$

Let U_γ stand for

$$\lambda u \left[\{ \{u\}(A_\gamma) \} (N_r) \right]$$

where $r = \xi(0)$; then, for all integers n ,

$$\{U_\gamma\}(N_n) \text{ conv } N_{\xi(n)}.$$

{265} It may be proved that there is a formula V such that

$\{ \{V\}(N_{\xi(n+1)}) \}$	conv
	N_1
	if, in going from the n -th to the $(n+1)$ -th complete
	conv configuration, the figure 0 is printed.

$$(N_{\xi(n)}) \quad \begin{cases} N_2 & \text{if the figure 1 is printed.} \\ \text{conv} & \text{otherwise.} \\ N_3 \end{cases}$$

Let W stand for

$$\lambda u \left[\left\{ \{V\}(\{A_\gamma\}(\{U_\gamma\}(u))) \right\}(\{U_\gamma\}(u)) \right]$$

so that, for each integer n ,

$$\left\{ \left\{ \{V\}(N_{\xi(n+1)}) \right\}(\xi(n)) \text{ conv } \{W_\gamma\}(N_n), \right.$$

and let Q be a formula such that

$$\left\{ \{Q\}(W_\gamma) \right\}(N_s) \text{ conv } N_{r(s)}$$

where $r(s)$ is the s -th integer q for which $(W_\gamma)(N_n)$ is convertible into either N_1 or N_2 .
Then, if M_γ stands for

$$\lambda w \left[\{W_\gamma\}(\left\{ \{Q\}(W_\gamma) \right\}(w)) \right]$$

it will have the required property.[\[16\]](#)

The Graduate College,
Princeton University,
New Jersey,
U.S.A.



[\[544\]](#) {Proc. London Math. Soc, Ser. 2, Vol. 43, No. 2198}

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO THE ENTSCHEIDUNGSPROBLEM. A CORRECTION

By A. M. Turing

In a paper entitled *On computable numbers, with an application to the Entscheidungsproblem* [17] the author gave a proof of the insolubility of the Entscheidungsproblem of the “engere Funktionenkalküls”. [18] This proof contained some formal errors which will be corrected here: there are also some other statements in the same paper which should be modified, although they are not actually false as they stand.

The expression for $\text{Inst}\{q_i S_j S_k Lq_l\}$ on p.260 of the paper quoted should read

$$(x, y, x', y') \left\{ (R_{S_j}(x, y) \ \& \ I(x, y) \ \& \ K_{q_i}(x) \ \& \ F(x, x') \ \& \ F(y', y)) \right. \\ \rightarrow \left(I(x', y') \ \& \ R_{S_k}(x', y) \ \& \ K_{q_l}(x') \ \& \ F(y', z) \vee [(R_{S_0}(x, z) \rightarrow (R_{S_0}(x', z)) \right. \\ \left. \left. \ \& \ (R_{S_1}(x, z) \rightarrow (R_{S_1}(x', z))) \ \& \ \dots \ \& \ (R_{S_M}(x, z) \rightarrow (R_{S_M}(x', z))) \right] \right) \left. \right\},$$

$S_0, S_1, \dots, S_M$ being the symbols which $\mathcal{M}$ can print. The statement on p261, line 33, viz.

$$\text{“Inst}\{q_a S_b S_d Lq_c\} \ \& \ F^{(n+1)} \rightarrow (CC_n \rightarrow CC_{n+1})$$

is provable” is false (even with the new expression for $\text{Inst}\{q_a S_b S_d Lq_c\}$): we are unable for example to deduce $F^{(n+1)} \rightarrow (-F(u, u''))$ and therefore can never use the term

$$F(y', z) \vee [(R_{S_0}(x, z) \rightarrow R_{S_0}(x', z)) \ \& \ \dots \ \& \ (R_{S_M}(x, z) \rightarrow R_{S_M}(x', z))]$$

{545} in $\text{Inst}\{q_a S_b S_d Lq_c\}$. To correct this we introduce a new functional variable G [$G(x, y)$ to have the interpretation “ x precedes y ”]. Then, if Q is an abbreviation for

$$(x) (\exists w) (y, z) \left\{ F(x, w) \ \& \ (F(x, y) \rightarrow G(x, y)) \ \& \ (F(x, z) \ \& \ G(z, y) \rightarrow G(x, y)) \right. \\ \left. \ \& \ [G(z, x) \vee (G(x, y) \ \& \ F(y, z)) \ \& \ (F(x, y) \vee F(z, y)) \rightarrow (-F(x, z))] \right\}$$

the corrected formula $\text{Un}(\mathcal{M})$ is to be

$$(\exists u) A(\mathcal{M}) \rightarrow (\exists s) (\exists t) R_{S_1}(s, t),$$

where $A(\mathcal{M})$ is an abbreviation for

$$Q \ \& \ (y) R_{S_0}(u, y) \ \& \ I(u, u) \ \& \ K_{q_1}(u) \ \& \ \text{Des}(\mathcal{M}).$$

The statement on p261 (line 33) must then read

$$\text{Inst}\{q_a S_b S_d Lq_c\} \& Q \& F^{(n+1)} \rightarrow (CC_n \rightarrow CC_{n+1})$$

and [line 29](#) should read

$$r(n, i(n)) = b, \quad r(n+1, i(n)) = d, \quad k(n) = a, \quad k(n+1) = c.$$

For the words “logical sum” on [p. 260, line 15](#), read “conjunction”. With these modifications the proof is correct. Un ($\mathcal{M}$) may be put in the form [\(I\)](#) (p.263) with $n = 4$.

Some difficulty arises from the particular manner in which “computable number” was defined ([p.233](#)). If the computable numbers are to satisfy intuitive requirements we should have:

If we can give a rule which associates with each positive integer n two rationals a_n, b_n satisfying $a_n \leq a_{n+1} < b_{n+1} \leq b_n, b_n - a_n < 2^{-n}$, then there is a computable number α for which $a_n \rho \alpha \leq b_n$ each n .

(A)

A proof of this may be given, valid by ordinary mathematical standards, but involving an application of the principle of excluded middle. On the other hand the following is false:

There is a rule whereby, given the rule of formation of the sequence a_n, b_n in (A) we can obtain a D.N. for a machine to compute α .

(B)

That (B) is false, at least if we adopt the convention that the decimals of numbers of the form $m/2^n$ shall always terminate with zeros, can be seen in this way. Let $\mathcal{M}$ be some machine, and define c_n as follows: $c_n = \frac{1}{2} - 2^{-m-3}$ if $\mathcal{M}$ has not printed a figure 0 by the time the n -th complete configuration is reached $c_n = \frac{1}{2} - 2^{-m-3}$ if 0 had first been printed as the m -th [\[546\]](#) complete configuration ($m \leq n$). Put $a_n = c_n - 2^{-n-2}, b_n = c_n + 2^{-n-2}$. Then the inequalities of (A) are satisfied, and the first figure of α is 0 if $\mathcal{M}$ ever prints 0 and is 1 otherwise. If (B) were true we should have a means of finding the first figure of α given the D.N. of $\mathcal{M}$: i.e we should be able to determine whether $\mathcal{M}$ ever prints 0, contrary to the results of [§8](#) of the paper quoted. Thus although (A) shows that there must be machines which compute the Euler constant (for example) we cannot at present describe any such machine, for we do not yet know whether the Euler constant is of the form $m/2^n$.

This disagreeable situation can be avoided by modifying the manner in which computable numbers are associated with computable sequences, the totality of computable numbers being left unaltered. It may be done in many ways [\[19\]](#) of which this is an example. Suppose that the first figure of a computable sequence γ is i and that this is followed by 1 repeated n times, then by 0 and finally by the sequence whose r -th figure is c_r ; then the sequence γ is to correspond to the real number

$$(2i - 1) n + \sum_{r=1}^{\infty} (2c_r - 1)(\frac{2}{3})^r.$$

If the machine which computes γ is regarded as computing also this real number then (B) holds. The uniqueness of representation of real numbers by sequences of figures is now lost, but this is of little theoretical importance, since the D.N.'s are not unique in any case.

The Graduate College,
Princeton, N.J.,
U.S.A.



Published on the abelard site by permission of the London Mathematical Society.
Originally published by the London Mathematical Society in *Proceedings of the London Mathematical Society*,
Series 2, Vol.42 (1936 - 37) pages 230 to 265,
with corrections from *Proceedings of the London Mathematical Society*,
Series 2, Vol.43 (1937) pages 544 to 546.

Endnotes

1. Gödel, "Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme, I", *Monatshefte Math. Phys.*, 38 (1931). 173-198.
2. Alonzo Church. "An unsolvable problem of elementary number theory", *American J of Math.*, 58(1936), 345 – 363.
3. Alonzo Church. "A note on the Entscheidungsproblem", *J. of Symbolic logic*, 1 (1930), 40 – 41.
4. In this reproduction, we at *abelard.org* are using a redrawn blackletter font in place of the High German blackletter fonts used in Turing's paper. The typeface used in the original paper makes it extremely difficult to systematically and fluently distinguish between letters, especially capital C, capital E and capital S. English and German blackletter typefaces have, fundamentally, an extremely similar character set. No doubt, however, they varied widely in detail between different printing presses. Our conclusion is that this minor modification to the original typesetting improves the readability of the paper, and thus conveys Turing's intent more effectively, without detracting from the artistry of his intended layout.
5. Cf. Hobson, *Theory of functions of a real variable* (2nd ed., 1921), 87, 88.
6. If we regard a symbol as literally printed on a square we may suppose that the square is $0 \leq x \leq 1, 0 \leq y \leq 1$. The symbol is defined as a set of points in this square, viz. the set occupied by printer's ink. If these sets are restricted to be measurable, we can define the "distance" between two symbols as the cost of transforming one symbol into the other if the cost of moving unit area of printer's ink unit distance is unity, and there is an infinite supply of ink at $x = 2, y = 0$. With this topology, the symbols form a conditionally compact space.
7. The expression "the functional calculus" is used throughout to mean the restricted Hilbert functional calculus.
8. It is most natural to construct first a choice machine (§2) to do this. But it then easy to construct the required automatic machine. We can suppose that the choices are always choices between two possibilities 0 and 1. Each proof will then be determined by a sequence of choices $i_1, i_2, \dots, i_n$ ($i_1 = 0$ or $1, i_2 = 0$ or $1, \dots, i_n =$

0 or 1), and hence the number $2n + i_1 2^{n+1} + i_2 2^{n-2} + \dots + i_n$, completely determines the proof. The automatic machine carries out successively proof 1, proof 2, proof 3,

9. The author has found a description of such a machine.
10. The negation sign is written before an expression and not over it.
11. A sequence of r primes is denoted by $(^r)$.
12. If $\mathcal{M}$ computes $\mathcal{M}$, then the problem whether γ prints 0 infinitely often is of the same character as the problem whether $\mathcal{M}$ is circle-free.
13. A function α_n may be defined in many other ways so as to run through the computable numbers.
14. Although it is not possible to find a general process for determining whether a given number is satisfactory, it is often possible to show that certain classes of numbers are satisfactory.
15. Loc. cit.
16. In a complete proof of the λ -definability of computable sequences it would be best to modify this method by replacing the numerical description of the complete configurations by a description which can be handled more easily with our apparatus. let us choose certain integers to represent the symbols and the m -configurations of the machine. Suppose that in a certain complete configuration the numbers representing the successive symbols on the tape are $s_1 s_2 \dots s_n$, that the m -th symbol is scanned, and that the m -configuration has the number t ; then we may represent this complete configuration by the formula

$$[[N_{s_1}, N_{s_2}, \dots, N_{s_{m-1}}, [N_t, N_{s_m}], [N_{s_{m+1}}, \dots, N_{s_n}]]$$
 where $[a, b]$ stands for $\lambda u [\{ \{ u \} (a) \} (b)]$,
 $[a, b, c]$ stands for $\lambda u [\{ \{ \{ u \} (a) \} (b) \} (c)]$,
 etc.
17. *Proc. London Math. Soc* (2) 42 (1936 – 7), 230 – 265.
18. The author is indebted to P. Bernays for pointing out these errors.
19. The use of overlapping intervals for the definition of real numbers is due originally to Brouwer.

The Turing test and intelligence gives a detailed logical analysis of the relationship between intelligence and Turing's proposed test of intelligence (as outlined in *Computing machinery and intelligence*).



email abelard@abelard.org

© abelard, 2001, 15 may

all rights reserved

Last updated 20.4.2004

the address for this document is <http://www.abelard.org/turpap2/turpap2.htm>

13,936 words

prints as 37 A4 pages (on my printer and set-up)

document abstracts	the mechanics of inflation	a.m.turing: computing machinery & intelligence	the Turing test and intelligence	Metalogic A: The Confusions of Gödel	site orientation	multiple use for this glittering en
-------------------------------	---	---	---	---	-----------------------------	--